双环路配送机器人交叉口通行与共享充电资源联合调度

EE6228 课程项目（M02 与 M04 联合选题）

纯软件二维在线仿真

摘要

四台配送机器人在两条单向环路上工作，两条环路共用一个互斥交叉口，并共用一个只能经由该交叉口到达的充电区。本项目构建了确定性离散事件仿真器、独立的调度验证器和浏览器回放界面，定义了归一化目标函数，并比较了四种启发式规则、一个同时决定任务分配、任务顺序、交叉口时隙、充电位选择和充电量的离线 CP-SAT 模型，以及它的在线滚动时域版本。在 700 次通过验证的运行（10 种设置 $\times$ 10 个随机种子 $\times$ 7 种策略）中，CP-SAT 相对先到先服务（FCFS）将目标值降低约 50 %（迟到时间减少 62 %，完工时间缩短 28 %），在线滚动版本降低约 37 %，而各单一规则基线之间相差不过几个百分点。我们还说明：公平比较要求所有方法在时域末端保留相同的电量储备；并讨论了归一化目标函数的局限。

1 案例介绍

两个配送区域 A 和 B 各由一条封闭的单向环路服务。两条环路在唯一的互斥交叉口 X 相交，两个区域共用一个充电区（入口 H，充电位 C1 和 C2），充电区只能经由 X 到达。网络中有四台虚拟机器人：R1 和 R2 服务区域 A，R3 和 R4 服务区域 B。取送任务随时间到达，每个任务有释放时间、取货站、送货站、装卸时间、截止时间和优先级。

两类共享资源相互耦合。需要充电的机器人必须两次通过 X（去和回），因此交叉口拥堵会推迟充电；充电排队反过来又会推迟该机器人以及排在其后的机器人此后的所有交叉口请求。电池约束使问题更难：机器人只有在完成任务后仍能在不低于安全阈值的情况下到达充电位时，才能接受该任务。本项目研究如何协调**任务分配、任务顺序、交叉口时隙、充电位选择和充电时长**，使机器人队伍在高负载下保持安全与准时。

范围。地图和数据均为自行设计；机器人沿固定的有向路线移动（不考虑连续运动学和避障）；电量、装卸和充电均为软件状态。研究重点是调度与资源冲突。

2 系统描述

2.1 网络

每条环路是有向回路 $X \to Z_1 \to Z_2 \to Z_3 \to Z_4 \to W_Z \to X$，其中 $Z \in \{A, B\}$（图 1）。W_A 和 W_B 是机器人等待通过交叉口的唯一位置。所有进入 X 的边（$W_A \to X$、$W_B \to X$、$H \to X$）都是一次 通过；X 同一时刻最多容纳一台机器人，相邻两次通过之间两侧各需保留安全间隔 β。充

电支路为 $X \to H \to C_p \to H \to X$；机器人在 H 排队，绝不在 X 内或已被占用的充电位内等待，充电结束后立即离开充电位。

由于机器人沿环路行驶，每绕一圈都会经过 X：交叉口不仅位于去充电的路上，也位于每个送货点在取货点“后方”的配送路径上，以及每次绕环的空驶调度路径上。

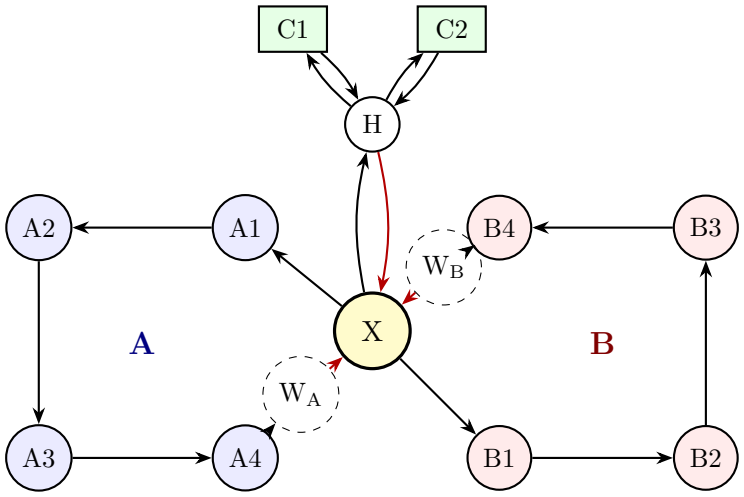


图 1: 实际实现的网络。红色箭头表示进入互斥交叉口 X 的通过；虚线节点为 X 前的等待点。R1/R2 从 A1 出发，R3/R4 从 B1 出发。

2.2　默认参数

参数	基础设置	测试范围
机器人数量	4 台（每条环路 2 台）	–
任务数量	40	20、40、60
初始电量	每台机器人 50–100 % 均匀随机（固定种子）	–
安全阈值 $S_{\min}$	10 %	10、20、30 %
普通边	20 m，20 s，2 %	边长可编辑
进入 X 的通过	6 s	5–8 s
安全间隔 β	每侧 2 s（两次通过之间 4 s）	1–4 s
等待耗电 ρ_w	0.2 %/min	0–0.5 %/min
装货 / 卸货	各 10–20 s，各耗电 0.5 %	–
充电速率 ρ_c	线性，5 %/min	3–8 %/min
充电位数量	2	1、2
截止时间	释放后 8–15 min（紧张档：4–8 min）	–
优先级	1（50 %）、2、3	紧急任务 4–5
区域 A 任务占比	50 %	1:1、2:1、3:1
紧急任务	0	+10 %、+20 %（3–6 min 内截止）

表 1: 默认参数（SOC：电池荷电状态，单位为百分点）。

n 个任务的释放时间在 $[0, 12n)$ 秒内均匀分布。生成器使用固定随机种子，紧急任务和初始电量来自独立的随机数流，因此增加紧急任务不会改变同一种子下的基础任务。所有策略都在完全相同的实例上评估。

2.3 目标函数

方案中的目标函数把单位不同的项相加：完工时间（秒）、按优先级加权的迟到时间（秒）、排队时间（秒）和能耗（百分点）。高负载时迟到项可达数万秒，会淹没其他所有项。因此，每一项都除以同一实例上 FCFS 调度的对应值，四个归一化项等权相加：

$$J = \frac{1}{4}\left(\frac{C_{\max}}{\hat{R}_C} + \frac{\sum_i p_i T_i}{\hat{R}_T} + \frac{W_{\text{cross}} + W_{\text{charge}}}{\hat{R}_W} + \frac{E}{\hat{R}_E}\right), \tag{1}$$

$$T_i = \max(0,\, C_i - d_i), \qquad \hat{R}_k = \max\left(R_k^{\text{FCFS}},\, f_k\right), \qquad f = (1\,\text{s},\ 60\,\text{s},\ 60\,\text{s},\ 1\,\%), \tag{2}$$

其中 $C_{\max}$ 为最后一个任务的完成时间，p_i 为优先级，C_i 和 d_i 为任务 i 的完成时间和截止时间，T_i 为其迟到时间，W 为交叉口与充电位的总排队时间，E 为总能耗。下限 f_k 只在 FCFS（几乎）没有迟到或排队时起作用。因此 FCFS 的得分为 $J = 1$（当其某一项低于下限时略小于 1），$J = 0.6$ 可理解为“在这一加权平衡下比 FCFS 好 40 %”。电池安全不参与权衡：低于安全阈值是硬约束，由独立验证器检查，因此所有报告的调度中 $N_{\text{unsafe}} = 0$。

3 仿真器

仿真器是一个不依赖第三方库的 Python 网页应用（标准库 HTTP 服务器，SVG 前端），核心是带预约日历的确定性离散事件引擎：

- **世界与能耗模型**（`simulator.py`、`customization.py`）：有向边带长度、行驶时间和能耗；移动、通过、等待、装卸和充电均按动作更新电量。
- **预约日历**：X 一个（含 2β 间隔），每个充电位一个。机器人申请资源时获得最早的可行时隙，等待时间作为显式的 `cross_wait` 或 `charge_wait` 动作记录在允许的等待点。
- **独立验证器**（`validate_result`）：仅根据返回的动作日志重新计算每个动作的能耗，并检查边的合法性、路径连续性、先取后送、释放时间、机器人载量、每台机器人的动作是否重叠、资源互斥及间隔（由物理轨迹推出，而不依赖预约标签）、等待位置、充电后立即离开以及电量上下界。未通过验证的调度绝不会作为结果返回。
- **回放与分析**（`static/`）：二维动画只读取事件日志（从不自行推算时间），另有甘特图、电量曲线、X/C1/C2 预约时间轴、配送表和策略对比；任务、调度、事件、机器人和汇总均可导出为 CSV。
- **命令模式**：可严格执行外部计划（机器人、开始时间、动作）；时间、资源和电量冲突会被报告，而不会被自动修复。

4 调度策略

4.1 基线策略

1. **FCFS（先到先服务）**——按释放顺序处理任务；每个任务交给预计最早完成的兼容机器人；电量低于 20 % 的机器人在下一个任务前先充电；交叉口时隙按预约顺序分配。

2. **分阶段**——方案中“先分配任务、再预约交叉口、最后安排充电”的分解。第一阶段按名义行驶时间做列表调度来分配任务，忽略 X、充电位和电量；第二阶段按交叉口预约执行每台机器人的固定序列；第三阶段只在下一个任务否则会不安全时插入充电。
3. **截止时间优先 + 预约**——最早截止时间优先（优先级打破平局），使用完整引擎的资源预约与电量检查。
4. **联合贪心**——对两台兼容机器人评估最多八个即将到来的任务，计入由此产生的交叉口和充电预约及充电插入，选出得分最低的候选：

$$\text{score} = C_i + 3\,p_i T_i + 0.5\,W^{\text{new}} + 2\,E^{\text{new}},$$

其中 W^{new} 和 E^{new} 为该候选新增的排队时间和能耗。

所有自动策略在接受任务前都保留电量储备：机器人必须能完成该任务、到达 H，并扛过当前已预约的充电排队以及为其他三台机器人预留的保守余量。所有启发式规则都固定充电到 95 %。

4.2 CP-SAT 联合优化

完整模型见 `docs/optimizer_model.md`，本节给出概要。关键观察是：路线固定，因此一台机器人任意两个相邻作业之间的行驶时间、能耗以及经过 X 的次数都可预先算出。每台机器人 k 对应一个图 $V_k = \{0\} \cup I_k \cup Q_k$，顶点包括表示当前状态的起点 0、其环路上的所有任务 I_k，以及可选的充电访问 Q_k。

决策变量。 $x_{ik} \in \{0,1\}$ 表示任务 i 分配给机器人 k；弧变量 $a_{kab} \in \{0,1\}$ 选择每个顶点的后继；s_i, c_i 为任务开始与完成时间；对每次通过 r，e_{kr} 为进入时间，$w_{kr} \geq 0$ 为在 X 前的等待，离开时间 $l_{kr} = e_{kr} + \tau_r$；对每次充电访问 q，有是否存在变量 y_{kq}、充电位选择变量 b_{kqp}、在 H 的排队时间 w^C_{kq}、充电开始时间 u_{kq} 与时长 D_{kq}；每个顶点在到达后和服务后的电量；迟到时间 T_i 与完工时间 $C_{\max}$。

约束。

$$\sum_k x_{ik} = 1 \quad \forall i, \qquad \textsc{Circuit}\big(\{a_{kab}\}\big) \quad \forall k \qquad \text{（分配）} \tag{3}$$

$$a_{kab} = 1 \;\Rightarrow\; s_b \geq c_a, \qquad s_i \geq r_i \qquad \text{（先后与释放）} \tag{4}$$

$$e_{kr} = \text{arrival}_{kr} + w_{kr}, \qquad l_{kr} = e_{kr} + \tau_r \qquad \text{（通过时间）} \tag{5}$$

$$\textsc{NoOverlap}\big(\{[e_{kr},\, l_{kr} + 2\beta)\}_{k,r}\big) \qquad \text{（X 互斥）} \tag{6}$$

$$u_{kq} = h_{kq} + w^C_{kq} + \tau_{Hp}, \qquad S^-_{kq} = S_{H,kq} - \rho_w w^C_{kq} - g_{Hp} \geq S_{\min} \qquad \text{（H 处排队）} \tag{7}$$

$$S^+_{kq} = S^-_{kq} + \rho_c D_{kq} \leq 100, \qquad \sum_p b_{kqp} = y_{kq} \qquad \text{（充电量）} \tag{8}$$

$$\textsc{NoOverlap}\big(\{[u_{kq},\, u_{kq} + D_{kq}) : b_{kqp} = 1\}\big) \quad \forall p \qquad \text{（充电位互斥）} \tag{9}$$

$$soc_{\text{out}} = soc_{\text{in}} - \sum g_{ab} - \sum g_h - \rho_w \sum w \;\geq\; S_{\min} \qquad \text{（能耗）} \tag{10}$$

$$soc_{\text{last}(k)} \geq S_{\min} + G_{k \to H \to p} + \rho_w\, \bar{W}_{\text{queue}} + \delta \qquad \text{（末端储备）} \tag{11}$$

其中 τ 和 g 为边的时长与能耗，g_h 为装卸能耗，h_{kq} 和 $S_{H,kq}$ 为到达 H 的时间与电量，$G_{k \to H \to p}$ 为从机器人最后一个站点到最近充电位的能耗，$\bar{W}_{\text{queue}} = 3\,(95 - S_{\min})/\rho_c$ 为启发式规则预留的“排

在另外三次完整充电之后”的等待时间，$\delta = 0.5\,\%$ 为小余量。每条弧上和每个任务内部的每次通过都是一个可选区间，只有选中对应弧时才存在；等待发生在进入前的等待点（W_A、W_B 或 H），绝不在 X 内。机器人充电结束后立即离开充电位，充电量 $\rho_c D_{kq}$ 是决策变量，而启发式规则固定充到 95 %。末端储备与启发式规则在接受任何任务前所检查的条件相同。若没有余量 δ，最优解会恰好落在边界上，随后被仿真器的检查拒绝，因为该检查在前往 H 的途中还会计入在 X 前排队的耗电。迟到时间与完工时间通过 $T_i = \max(0, c_i - d_i)$ 和 $C_{\max} = \max_i c_i$ 线性化（`AddMaxEquality`）。

目标函数正是式 (1) 中的归一化 J；其参考值 $\hat{R}_k$ 在求解前算出，是常数，因此 J 保持线性。时间以毫秒为单位、电量以整数单位表示，所有时长和能耗都向上取整，保证整数可行的调度在精确仿真器中仍然可行。模型以联合贪心的调度作为初始解（`AddHint`）。

解码。 求解器的决策（序列、通过进入时间、充电位、充电开始时间与目标电量）通过仿真器自己的规划器重放，因此返回调度中的时间和能耗都来自仿真器，且结果必须通过独立验证器。若解码后的调度比初始解差或未通过验证，则返回初始解并记录回退，因此 CP-SAT 报告的调度绝不会比联合贪心差。

搜索。 CP-SAT 使用 8 个并行工作线程。只用一个线程时，求解器在 40 个任务的实例上运行约两分钟也没有改进初始解；而并行组合（主要由大邻域搜索线程完成改进）在相同时间内显著改进了它。在小实例（4–6 个任务）上模型可证明最优，并与解析下界一致（单元测试）。在实际规模上下界仍然较弱，因此结果是优质的可行解，而非已证明的最优解。

4.3 滚动时域再优化

离线优化假设所有任务在时刻 0 已知。滚动策略去掉了这一假设：任务只在其释放时刻才可见（算法 1）。

算法 1 滚动时域 CP-SAT

$t \leftarrow 0$
当 仍有未完成任务 **执行**
  $\mathcal{K} \leftarrow$ 截至 t 已释放且尚未开始的任务
  $\mathcal{W} \leftarrow \mathcal{K}$ 中按紧急程度排序的任务，直到每条环路约 600 s 的工作量
  $\hat{R} \leftarrow$ 从当前状态继续、仅针对 $\mathcal{K}$ 的 FCFS 参考值 ▷ 在线归一化
  plan $\leftarrow$ 从机器人当前状态出发、对 $\mathcal{W}$ 求解 CP-SAT，已承诺的 X 与充电位预约固定（1 个线程，确定性预算约 0.5 s，初始解为联合贪心）
  承诺 plan 中所有在 $t + 60\,\mathrm{s}$ 之前开始的作业，并冻结其预约
  $t \leftarrow t + 60\,\mathrm{s}$
结束循环

已承诺的动作不再修改。每个窗口求解都要求每台机器人最后计划的作业之后保留末端储备，使下一个窗口不会陷入电量死局。归一化本身也是在线的：参考值只来自截至当前已知任务的 FCFS 调度。早期版本使用了全实例参考值，代码审查发现这会改变决策，即泄露了未来信息。最终调度与其他策略一样，用全实例的 J 评分。

5 实验

5.1 实验设置

方案中的实验计划由 `experiments.py` 执行。每种配置使用种子 1–10；对同一种子，全部七种策略（六种策略加上第 6.2 节的消融实验）得到完全相同的任务列表、配置和固定种子的初始电量。基础设置为 40 个任务、2 个充电位、10 % 安全阈值、区域任务比例 1:1、无紧急任务；每个实验只改变一个因素（表 2）。

实验	设置
算法比较	基础设置
任务负载	20、40、60 个任务
充电资源	1 或 2 个充电位
区域不平衡	A:B = 1:1、2:1、3:1
安全阈值	10、20、30 %
动态任务	增加 0、10、20 % 紧急任务（释放后 3–6 min 截止，优先级 4–5）

表 2: 实验计划。

在多个实验中重复出现的相同实例（即基础设置）只求解一次，共计 10 种不同设置 × 10 个种子 × 7 种策略 = 700 次运行。每次运行记录 J 及其四个原始项、迟到任务比例、X 与充电位的排队时间、能耗、充电次数和运行时间。由于不同种子的难度差别很大，表中还给出同一实例上 J 相对 FCFS 的配对变化。

计算环境。四种启发式规则和滚动策略都是确定性的（滚动求解受 CP-SAT 确定性时间约束，而非墙钟时间），因此其调度与机器无关；这一点已通过在两台不同机器上逐位复现一次滚动运行得到验证。CP-SAT 受墙钟时间约束（120 s，8 个线程），因此依赖硬件；所有 CP-SAT 运行都使用同一机型（Google Colab 高内存 CPU 虚拟机，8 vCPU Intel Xeon @ 2.20 GHz，OR-Tools 9.15），每台虚拟机同一时间只运行一个求解。

公平性说明。启发式规则和离线 CP-SAT 预先知道全部任务列表（但不会在释放前开始任何任务）；滚动策略是唯一真正在线的方法。所有策略在时域末端保留相同的电量储备（每台执行任务的机器人都必须能排在满队列之后到达充电位；没有任务的机器人在任何策略下都不移动）。第一次运行的离线 CP-SAT 没有该储备，作为消融实验 `cpsat-nores` 保留；它常常以接近安全阈值的电量结束，并经常完全不充电，从而夸大了优势，第 6.2 节对此做了量化。启发式规则总是充到 95 %，而 CP-SAT 自行选择充电量。

5.2 算法比较（基础设置）

策略	J	相对 FCFS 均值 / 中位数	迟到比例	迟到时间 (s)	完工时间 (s)	充电次数	能耗 (%)	运行时间[a]
FCFS	1.000	–	52 %	26 028	3 233	5.0	443	< 0.1 s
分阶段	1.067	+7 / +2 %	51 %	27 264	3 441	5.4	449	< 0.1 s
截止时间优先 + 预约	1.003	0 / −4 %	51 %	25 994	3 210	4.9	438	< 0.1 s
联合贪心	0.793	−21 / −23 %	37 %	17 826	3 021	3.9	386	0.3 s
滚动 CP-SAT（在线）	0.636	−36 / −37 %	37 %	15 651	2 741	4.3	389	49 s
CP-SAT	**0.468**	**−53** / **−54** %	33 %	9 934	2 317	4.2	359	121 s
CP-SAT 无储备（消融）	0.349	−65 / −64 %	23 %	5 032	1 732	2.0	316	121 s

表 3: 基础设置（40 个任务、2 个充电位、10 % 阈值、1:1 区域）；10 个种子的均值，J 相对 FCFS 的配对变化以“均值 / 中位数”给出。[a] 每个实例实测的墙钟时间（CP-SAT 预算为 120 s；滚动策略在 2 vCPU 虚拟机上运行）。完整表格见 `results/results.md`。

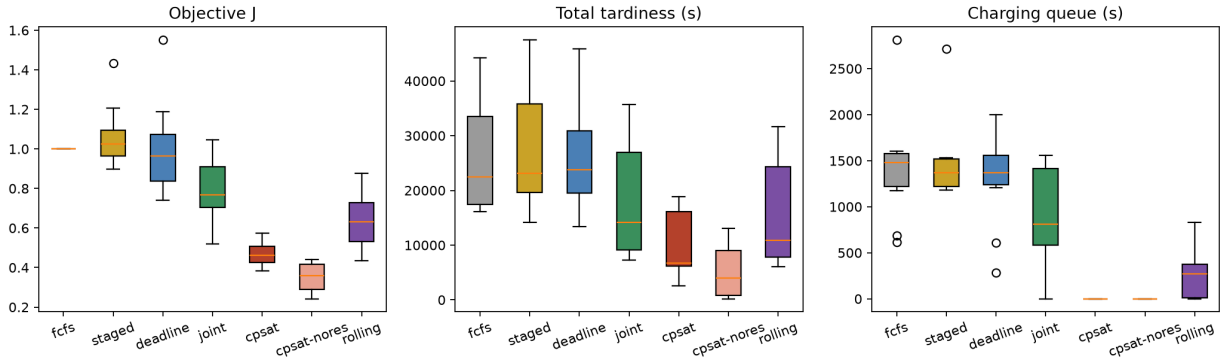


图 2: 基础设置，10 个种子：各策略的目标值 J、总迟到时间和充电排队时间。

全部 700 次运行都得到了通过验证、电量安全的调度；没有任何策略在任何实例上失败。均值和中位数给出相同的性能层级，并在大多数单个种子上成立（图 2）：三种单一规则基线彼此相差不超过 ±7 %（其内部顺序有所不同：按均值 FCFS 略优于截止时间优先，按中位数截止时间优先领先 4 %），联合贪心约好五分之一，滚动策略约好三分之一，离线 CP-SAT 约好一半。离线 CP-SAT 从未回退到初始解，但最优性间隙仍然较大（基础设置：中位数 58 %，范围 51–70 %；全部 100 个实例：中位数 63 %，范围 18–76 %）：回路模型的下界较弱，因此这些是优质可行解，而非已证明的最优解。

5.3 敏感性实验

表 4 给出 J 相对 FCFS 的配对变化中位数。均值见 `results/results.md`；在 FCFS 某项接近零而放大个别运行的情况下（第 6.3 节），均值与中位数差别很大，例如 20 个任务时分阶段和截止时间优先的均值分别为 +81 % 和 +134 %，而中位数为 +7 % 和 +10 %。

实验	设置	分阶段	截止时间优先	联合贪心	滚动	CP-SAT
任务负载	20 个任务	+7	+10	−30	−43	−62
	40 个任务	+2	−4	−23	−37	−54
	60 个任务	+8	+10	−16	−18	−47
充电位	1 个	+45	−1	−20	−28	−60
	2 个	+2	−4	−23	−37	−54
区域不平衡 A:B	2:1	+4	+2	−15[b]	−25	−48
	3:1	0	−6	−29	−30	−47
安全阈值	20 %	+2	+11	−18	−16	−51
	30 %	+17	−1	−13	−16	−52
紧急任务	+10 %	+1	−3	−12	−24	−52
	+20 %	+4	−5	−8	−20	−50

表 4: J 相对 FCFS 的配对变化中位数（%），每格 10 个种子。[b] 均值为 +19 %：有一个种子的 $J = 3.7$，尽管联合贪心在该实例上的完工时间、迟到和能耗都优于 FCFS（第 6.3 节）。

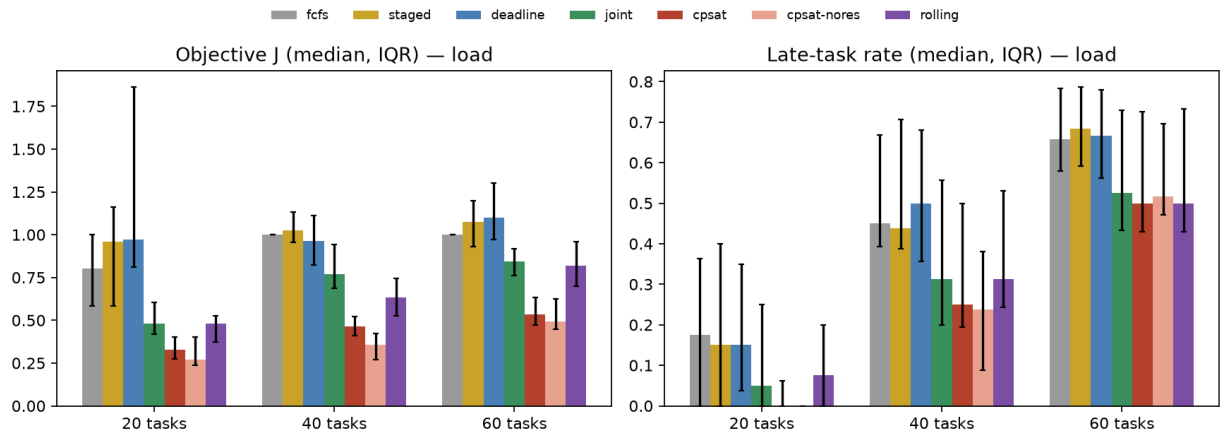


图 3: 任务负载实验：J 与迟到任务比例的中位数及四分位距。

- **任务负载。**随着任务增加，所有方法都变差，但启发式规则和滚动策略相对 FCFS 的优势缩小（−43 % → −18 %），而 CP-SAT 保持在 −47 % 到 −62 %（图 3）。60 个任务时每种方法都有一半以上的任务迟到（迟到比例 54–69 %）；队伍已超出能力，充电（7–10 次，每次最长 17 min）成为瓶颈资源。
- **单个充电位。**FCFS 的充电排队从 1 437 s 增加到 4 001 s，分阶段基线大幅恶化（+45 %）：它最后才决定充电，机器人会同时到达唯一的充电位。联合贪心和 CP-SAT 把充电与任务一起决定，保持了优势；CP-SAT 改进最多（−60 %）。
- **区域不平衡。**环路 A 的任务越多，所有方法的完工时间越长，因为 R1/R2 成为瓶颈。性能层级不变（CP-SAT 最好，其次是滚动和联合贪心，最后是单一规则基线），只是在基线层级内部，2:1 时截止时间优先按中位数略落后于 FCFS（+2 %）。
- **安全阈值。**阈值越高，所有方法都需要更多充电。CP-SAT 几乎不受影响（−51 % / −52 %），而

滚动策略退回到联合贪心的水平（−16 %）：它的每个窗口都必须以完整储备结束，在 20–30 % 阈值下这会迫使其提前充电，而离线视角本可避免。

- **紧急任务。**截止期短、优先级高的任务会使所有方法变差；离线 CP-SAT 保持约 −50 %，滚动策略为 −20 % 到 −24 %。对紧急任务最直观的截止时间优先规则只改进 3–5 %。

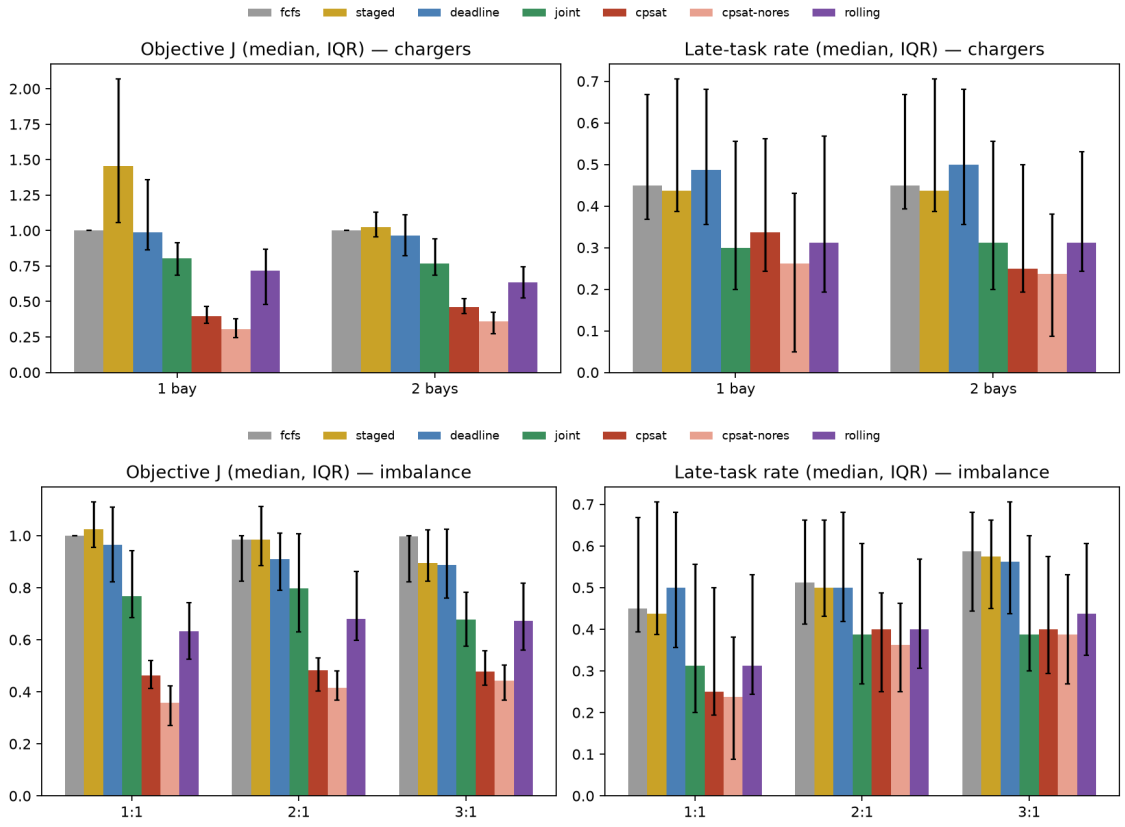


图 4: 充电位与区域不平衡实验（中位数，四分位距）。

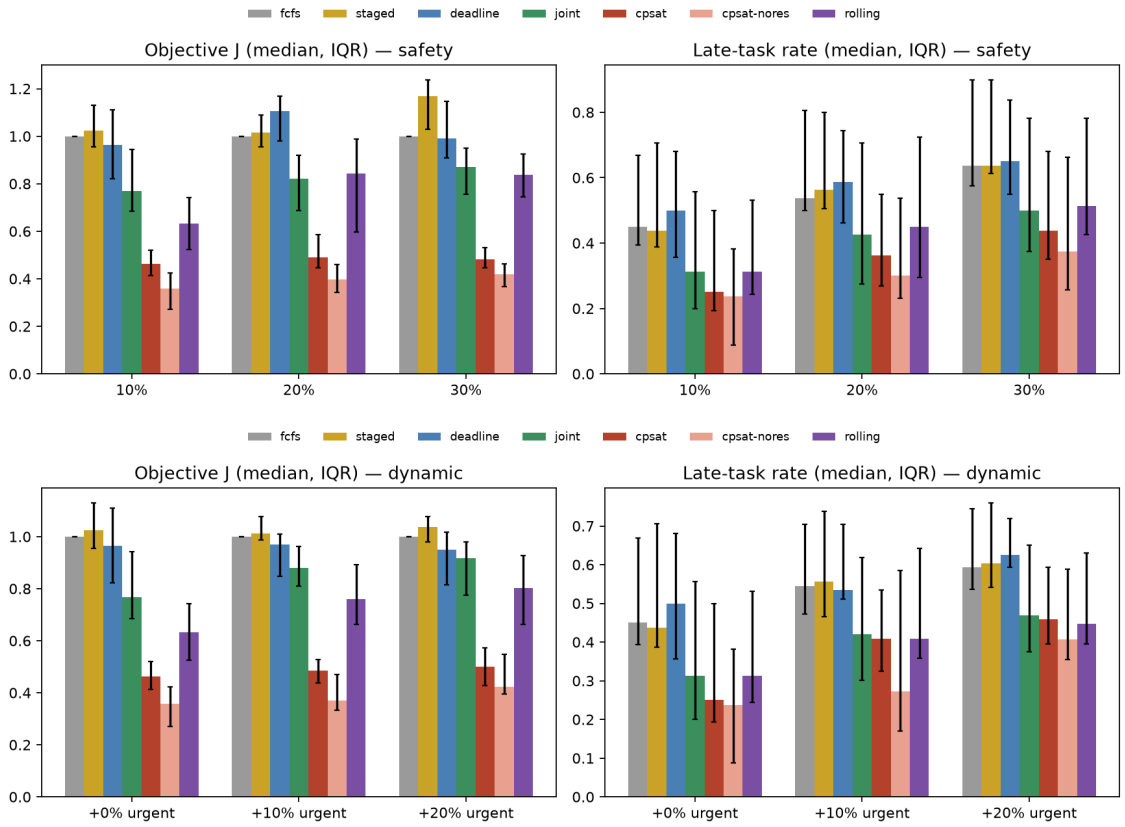


图 5: 安全阈值与紧急任务实验（中位数，四分位距）。

6 讨论

6.1 改进从何而来?

把 J 分解为四个归一化项（基础设置，平均贡献；FCFS 每项贡献 0.25）可以看出，各方法的改进方式不同（表 5）。

策略	$C_{\max}$	迟到	排队	能耗	J	J'^{c}
FCFS	0.250	0.250	0.250	0.250	1.00	1.00
联合贪心	0.233	0.182	0.160	0.218	0.79	0.84
滚动	0.211	0.157	0.048	0.219	0.64	0.78
CP-SAT	0.178	0.088	0.000	0.202	0.47	0.62
CP-SAT 无储备	0.132	0.039	0.000	0.178	0.35	0.47

表 5: 各归一化项对 J 的平均贡献（基础设置）。[c] $J' = (C_{\max} + \text{迟到} + \text{能耗})/0.75$，即去掉排队项后对其余三项重新加权的 J。

两种 CP-SAT 方法都几乎消除了排队（CP-SAT 平均 5.6 s，且全部来自一个种子；启发式规则在 X 处为 23–37 s，在充电位为 0.9–1.5 千秒）。机器人仍然会等待，但等待发生在出发前最后一个站点的断电待机状态，而不是在 X 前或 H 处。在模型中，待机不耗电，而排队会耗电并计入 J，因此优化器会推迟出发直到资源空闲；启发式规则则是先预约时隙再开到等待点。这在模型内

是合理的，但 CP-SAT 在 J 上的改进约有一半来自排队项。去掉该项后性能层级不变（表 5 中的 J'），CP-SAT 仍明显最优：相对 FCFS 总迟到时间减少 62 %、完工时间缩短 28 %（相对联合贪心分别为 44 % 和 23 %）。

迟到减少的一个合理的主要来源是充电：CP-SAT 的充电次数与联合贪心相近（4.2 次对 3.9 次），但它自行选择充多少以及何时充，而启发式规则总是以 5 %/min 的速度充到 95 %，每次最长 17 分钟，且常常发生在繁忙时段。CP-SAT 同时还改变了任务分配、顺序和出发时间，因此实验并未单独分离出这一效应；固定 95 % 目标的 CP-SAT 运行才是对照实验。

6.2 时域末端的电量储备

最初的 CP-SAT 实验不要求最后一个任务之后保留电量。优化器因此常常以接近安全阈值的电量结束，并在初始电量足够时跳过充电：在所有 20 个任务的实例上它从未充电，在 40 个任务时充电次数只有启发式规则的一半（2.0 次对 3.9–5.4 次）。启发式规则则总是保留足以排在满队列之后到达充电位的电量。对 CP-SAT 施加同样的储备后，40 个任务时其 J 从 0.349 升至 0.468，但在 60 个任务时几乎没有影响（中位数 -51 % 对 -47 %），因为此时机器人无论如何都必须充电。因此，在中等负载下离线 CP-SAT 表面优势中约五分之一（65 个百分点中的 12 个）来自时域末端效应。任何把离线方法与持续运行策略比较的研究都需要明确的末端条件。

第二个较小的教训：优化器把末端电量恰好放在储备边界上，而仿真器的储备检查在前往 H 的途中还计入了在 X 前排队的耗电，因此拒绝了许多这样的计划，使其回退到初始解（开发过程中，一次部分运行的 115 行结果中有 32 行回退，部分实例在多个实验中重复计数；这些行已被丢弃）。加入 $\delta = 0.5$ % 的余量后，最终 100 个实例中没有一个回退。模型与仿真器的精确一致，恰恰在优化器逼近约束边界时最为重要。

6.3 归一化目标函数

按 FCFS 归一化达到了目的：否则按优先级加权的迟到时间（数万秒）会比其他项大两个数量级，而归一化后四项的贡献相当。代价是在 FCFS 某一项接近零的实例上结果不稳定。在一个 2:1 实例上，FCFS 只排队了 18 s，于是排队项被除以 60 s 的下限，联合贪心 735 s 的排队单独就为其 J 增加了 $0.25 \times 735/60 \approx 3.06$（总计 3.7），尽管联合贪心在完工时间、迟到和能耗上都优于 FCFS。同样的效应使 20 个任务时分阶段和截止时间优先的均值大于 1（中位数为 0.96–0.97）。因此全文报告中位数和配对比较。更稳健的做法是用固定的、与实例无关的尺度归一化（例如整个实验的均值），或用该实例上所有策略的中位数归一化。

6.4 在线与离线

滚动策略是唯一不知道未来任务的方法，但在基础设置下它仍优于所有预知未来任务的启发式规则（中位数 -37 % 对联合贪心的 -23 %），并且在 20 个任务、单个充电位、2:1 区域不平衡和紧急任务设置下比联合贪心领先 8–14 个百分点。它的弱点正是滚动时域方法的典型问题：在 20–30 % 的安全阈值下，每个窗口的储备要求迫使其提前充电；在 60 个任务或 3:1 不平衡时，短视的前瞻无法绕开瓶颈，因此只与联合贪心持平（在 20 % 阈值下略落后）。它的每次重规划受确定性求解工作量约束（在 2 vCPU 机器上每次约 1 s，一整天 40 个任务共 49 s），因此可以实时运行。在 OR-Tools 版本相同、单线程且墙钟上限不起作用的条件下，其调度在不同机器上逐位一致（已在两台机器上验证）；运行时间自然不同。

6.5 局限性

- CP-SAT 结果依赖硬件（墙钟时间预算）；所有结果都在同一机型上得到。
- 最优性间隙较大；要声称接近最优，需要更好的下界（例如按环路的松弛）。
- 启发式规则能看到所有未来任务，这相对滚动策略对其有利，但相对离线 CP-SAT 则不然。
- 充电模型是线性的，网络只有一个交叉口，机器人沿固定路线移动且不考虑运动学。

7 结论

本项目为共用一个交叉口和一个充电区的两条配送环路构建了仿真器，配有独立验证器、浏览器回放以及在相同固定种子实例上比较六种策略（另加一个消融实验）的实验流程。在 700 次通过验证的运行中：

1. **联合决定任务、交叉口时隙和充电是值得的。**在基础设置下，各单一规则基线（FCFS、截止时间优先、分阶段）彼此相差不过几个百分点，联合贪心启发式改进约 20 %，而在一个模型中优化全部决策的 CP-SAT 在 J 上改进约 50 %（相对 FCFS 迟到时间减少 62 %、完工时间缩短 28 %）。在所有敏感性设置下，CP-SAT 保持在 $-47\,\%$ 到 $-62\,\%$（中位数），联合贪心在 $-8\,\%$ 到 $-30\,\%$ 之间变化，而分阶段基线可能比 FCFS 差很多（单个充电位时 $+45\,\%$）。
2. **充电是瓶颈。**高负载下限制队伍的是充电位而不是交叉口：对每种启发式规则，每次运行在 X 处的平均排队都不到一分钟，而充电排队可达数千秒。选择充多少、何时充是 CP-SAT 迟到更少的最合理解释（尚未通过实验单独分离），而把充电决策与任务分配分开（分阶段基线）在单个充电位时表现最差。
3. **在线再优化是可行的。**滚动 CP-SAT 在不知道未来任务的情况下，在中等负载时优于所有启发式规则，每次重规划约需一秒；在高负载或高安全阈值下它只与联合贪心启发式持平。
4. **公平比较需要谨慎。**要求相同的时域末端电量储备显著改变了 CP-SAT 的结果（J：$0.35 \to 0.47$），而归一化目标函数必须结合中位数解读，因为接近零的参考项可能主导它。